

Introduction to Deep Learning

Advanced Machine Learning

Units

- 5.1 Deep Learning fundamentals
- 5.2 Neural Network fundamentals
- 5.3 Deep Learning architectures
- 5.4 Practical Deep Learning

OVERVIEW : MODULE 05

Introduction to Deep Learning

Learning Outcomes

- Understand the fundamental principles of deep learning and its applications in various domains
- Differentiate between deep learning and traditional machine learning algorithms
- Gain proficiency in building and training basic deep learning models using popular frameworks like TensorFlow or PyTorch
- Explore advanced deep learning concepts such as convolutional neural networks (CNNs), recurrent neural networks (RNNs), and LSTM
- Acquire practical skills through hands-on projects, culminating in the implementation of a deep learning model to solve a specific problem
- Prepare a foundation for further exploration and study in advanced topics within the field of deep learning



5.1

Deep Learning Fundamentals

Lesson Learning Outcomes

- ✓ Understand the definition and significance of deep learning
- ✓ Differentiate between deep learning and traditional machine learning
- ✓ Gain familiarity with neural networks and their basic components

LESSON OVERVIEW

Module 5 Introduction to Deep Learning

Lesson 5.1 Deep Learning Fundamentals

This lesson provides an introductory understanding of **Deep Learning**, its significance in modern applications, and how it differs from traditional machine learning approaches.

We will explore the basic concepts of deep learning, including neural networks, and lay the foundation for subsequent lessons.

- Definition of Deep Learning
- Difference between machine learning and deep learning
- Overview of neural networks

Deep Learning Fundamentals

What is Deep Learning?

Deep Learning is a subset of machine learning that employs artificial neural networks with multiple layers to learn representations of data.

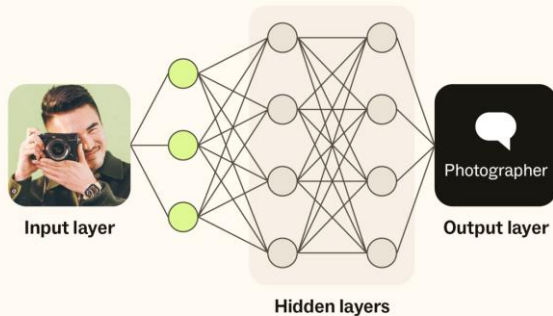
These representations enable the model to perform **complex tasks** such as **classification**, **regression**, and **pattern recognition**.

Deep learning offers some distinct advantages for certain tasks, especially those that involve complex data like images, text, or speech. Some advantages are as follows :

- **Automatic Feature Extraction**
- **Better Handling of Complex Data**
- **Improved Accuracy**

The deep learning process

The algorithm receives data, runs it through the input and hidden layers, and generates an output.



Deep Learning Fundamentals

Importance of Deep Learning

Applications

Deep learning has revolutionized various fields, including ,

- Computer Vision
- Natural language processing
- Speech recognition
- Healthcare
- Finance
- Autonomous driving

Deep Learning Fundamentals

Importance of Deep Learning

Advantages

Deep learning models can,

- Automatically learn hierarchical features from raw data
- Eliminating the need for manual feature engineering
- Handle large-scale unstructured datasets more effectively than traditional machine learning algorithms.

Deep Learning Fundamentals

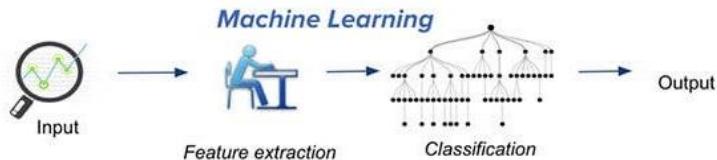
Deep Learning vs. Machine Learning

Differences

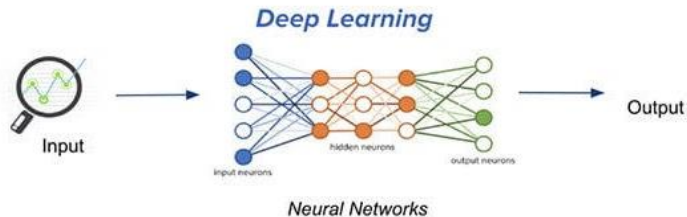
- Machine learning focuses on algorithms that can learn from data and make predictions
- Deep learning specifically refers to neural networks with multiple layers

Similarities

- Both involve learning from data to make predictions or decisions
- Both are used in various applications across industries



Traditional machine learning uses hand-crafted features, which is tedious and costly to develop.



Deep learning learns hierarchical representation from the data itself, and scales with more data.

Deep Learning Fundamentals

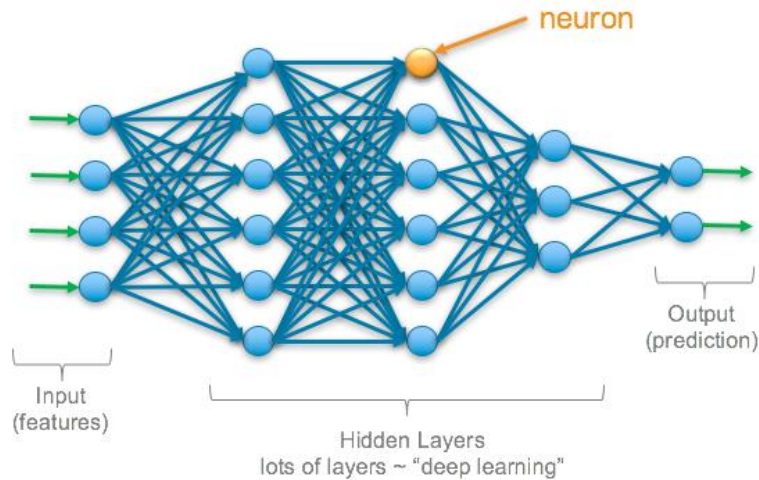
Overview of Neural Networks

Neurons and Activation Functions

- **Neurons** are basic computational units that receive inputs, perform a computation, and produce an output.
- **Activation functions** introduce non-linearity into the neural network, allowing it to learn complex patterns.

Layers

- **Input Layer:** Receives input data
- **Hidden Layers:** Intermediate layers where computations are performed
- **Output Layer:** Produces the final output of the network



Deep Learning Fundamentals

SUMMARY

- ✓ Deep learning is a subset of machine learning that utilizes neural networks with multiple layers.
- ✓ It offers advantages such as automated feature learning and is widely used across various industries.
- ✓ Neural networks consist of neurons, activation functions, and layers, which enable them to learn complex representations from data.

5.2

Neural Networks Fundamentals

Lesson Learning Outcomes

- ✓ Understand the basic structure and functioning of neural networks
- ✓ Learn about the role of neurons and activation functions in neural networks
- ✓ Gain familiarity with the processes of forward propagation and backpropagation for training neural networks

LESSON OVERVIEW

Module 5 Introduction to Deep Learning

Lesson 5.2 Neural Networks Fundamentals

This lesson delves into the foundational concepts of **Neural Networks**, covering their structure, functionality, and training processes.

We will explore the **basic components** of neural networks, including neurons, activation functions, and the **principles** of forward propagation and backpropagation.

- Artificial Neural Network, perceptron and activation function
- Layers, forward propagation and backpropagation

Neural Networks Fundamentals

Artificial Neural Networks (ANNs)

Artificial Neural Networks (ANNs) are computational models inspired by the structure and functioning of biological neural networks in the human brain.

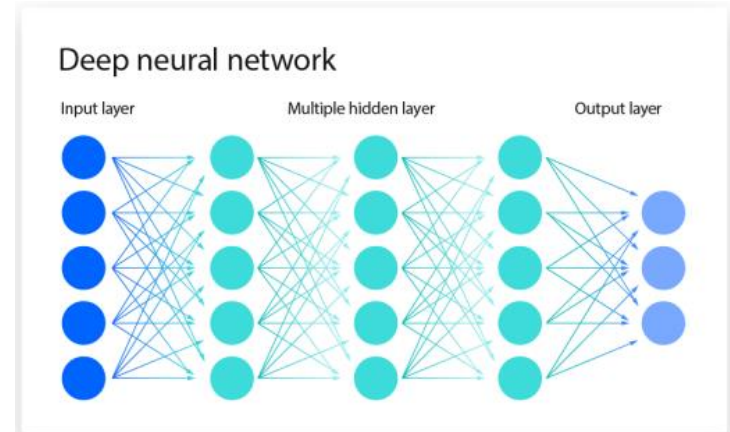
They consist of interconnected nodes, called neurons, organized in layers.

Structure

Input Layer: Receives input data.

Hidden Layers: Intermediate layers where computations are performed.

Output Layer: Produces the final output of the network.



Neural Networks Fundamentals

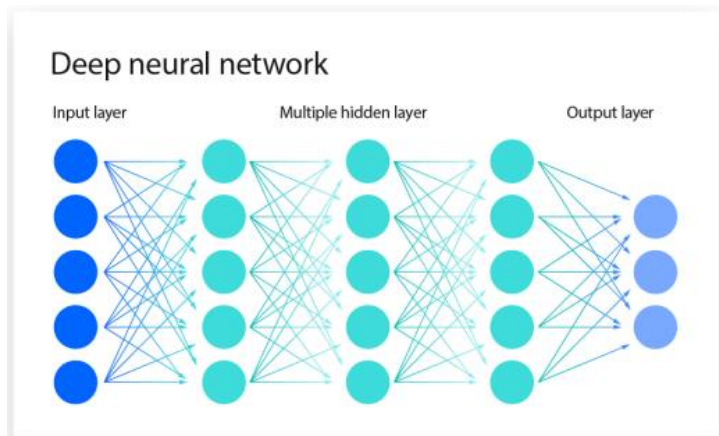
Artificial Neural Networks (ANNs)

Functioning

- Neurons in each layer take inputs, perform a computation (usually a weighted sum) apply an activation function, and produce an output
- The outputs from one layer serve as inputs to the next layer, propagating information through the network

Training

ANNs learn from examples by **adjusting** the weights of connections between neurons during a process called training. This is typically done using **optimization** algorithms like gradient descent.



Neural Networks Fundamentals

Perceptron

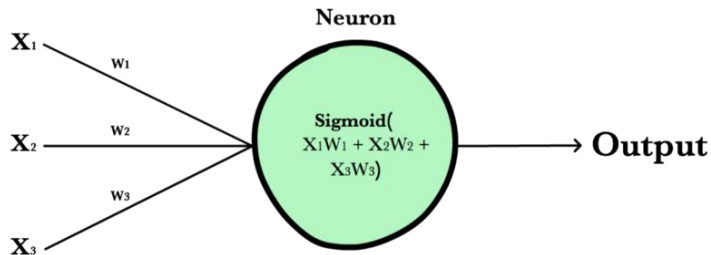
Perceptrons are the building blocks of artificial neural networks, representing a simplified model of a biological neuron. They take multiple inputs, each multiplied by a weight, sum them up, and apply an activation function to produce an output.

Structure

Inputs: The inputs to a perceptron represent features or attributes of the input data.

Weights: Each input is associated with a weight, which determines its contribution to the output.

Activation Function: The weighted sum of inputs is passed through an activation function, which determines the output of the perceptron.



Neural Networks Fundamentals

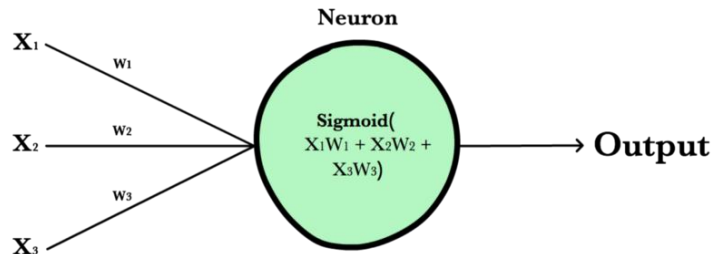
Perceptron

Activation Function

- The activation function enables the model to learn complex patterns in the data
- Common activation functions include step function, sigmoid function, hyperbolic tangent function (tanh), and rectified linear unit (ReLU)

Training

- **Perceptrons** are trained using a process called supervised learning, where they adjust their weights based on the error between the predicted output and the actual output
- The weights are updated iteratively using an **optimization algorithm** such as gradient descent to minimize the error
- **Loss Function** measures the difference between predicted and actual values. For example Mean Squared Error (MSE), RMSE etc.



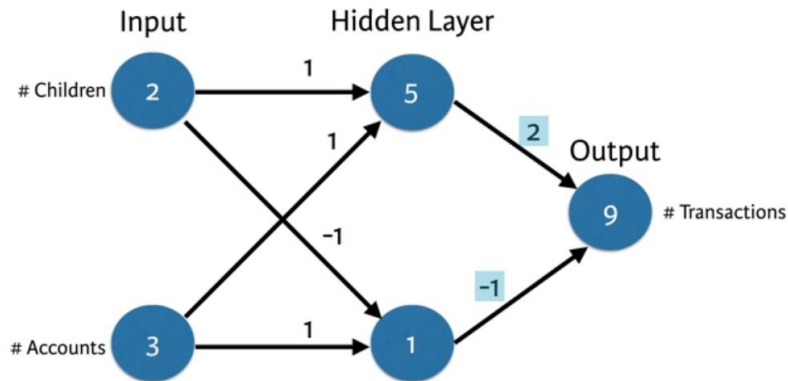
Neural Networks Fundamentals

Forward Propagation

Forward propagation is the process by which data flows through the neural network, from the input layer to the output layer.

During forward propagation, each neuron **receives input signals**, **applies weights**, **computes the weighted sum**, and **passes the result through an activation function to produce an output**.

Forward propagation

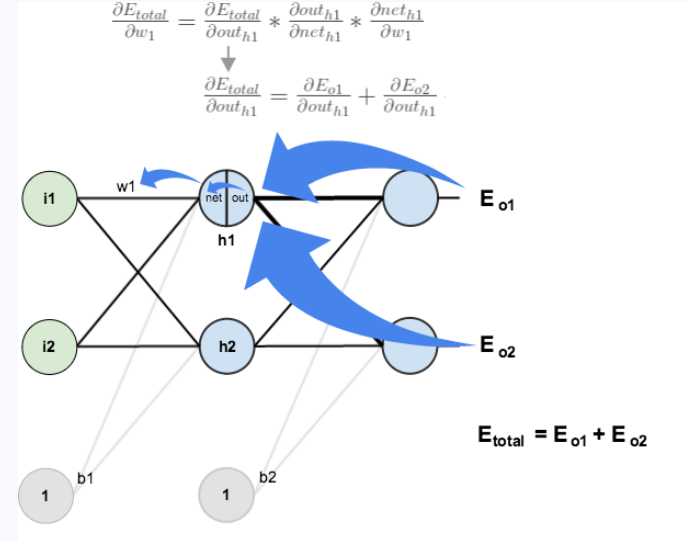


Neural Networks Fundamentals

Backpropagation

Backpropagation is the process of updating the weights of the neural network based on the error between the predicted output and the actual output.

It involves computing the gradient of the loss function with **respect to each weight in the network** and **adjusting the weights accordingly to minimize the error**.



Introduction to Common Supervised Learning Algorithms

SUMMARY

- ✓ Neural networks consist of neurons, activation functions, and layers, which enable them to learn complex patterns from data.
- ✓ Forward propagation involves passing data through the network to produce predictions, while backpropagation is used to update the network's weights based on prediction errors.



5.3

Deep Learning Architectures

Lesson Learning Outcomes

- ✓ Understand the principles and applications of Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Long Short-Term Memory (LSTM) networks
- ✓ Differentiate between the architectures and identify suitable use cases for each
- ✓ Gain insight into how deep learning architectures are applied in real-world scenarios such as image recognition, natural language processing, and time series analysis

LESSON OVERVIEW

Introduction to Deep Learning

Lesson 5.3 Deep Learning Architectures

This lesson explores various deep learning architectures, including **Convolutional Neural Networks** (CNNs), **Recurrent Neural Networks** (RNNs), and **Long Short-Term Memory** (LSTM) networks.

We will discuss the structure, functionality, and applications of each architecture in solving specific types of problems.

- Convolutional Neural Network
- Recurrent Neural Network
- Long Short-Term Memory (LSTM)
- Comparison and application of different architectures

Deep Learning Architectures

Convolutional Neural Networks (CNNs)

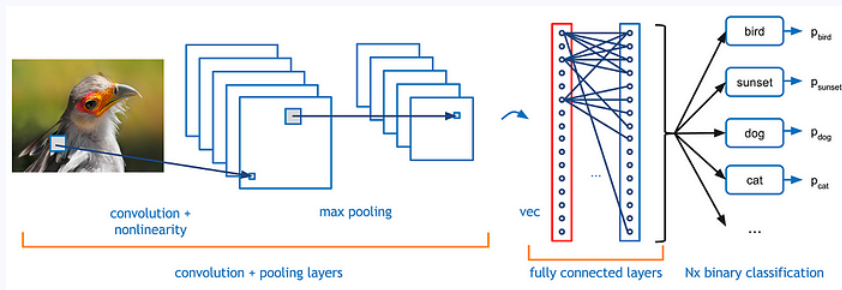
Convolutional Neural Networks (CNNs) are deep learning architectures specifically designed for processing structured grid-like data, such as images.

Structure

CNNs consist of convolutional layers, pooling layers, and fully connected layers.

Applications

CNNs are widely used in image recognition, object detection, and image segmentation tasks.



Deep Learning Architectures

Recurrent Neural Networks (RNNs)

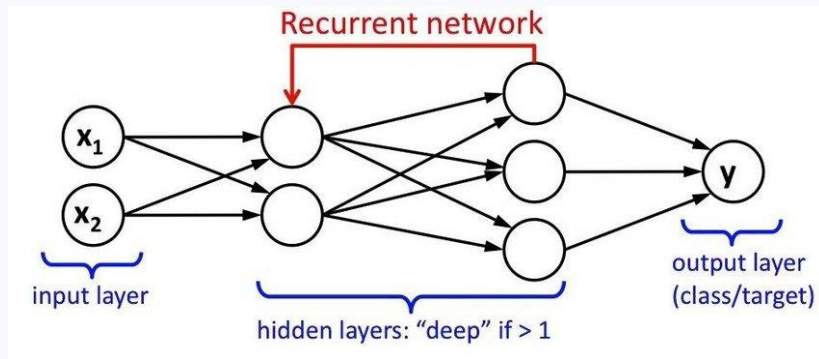
Recurrent Neural Networks (RNNs) are designed to handle sequential data by maintaining an internal state or memory.

Structure

RNNs have recurrent connections that allow information to persist over time steps, enabling them to process sequences of variable length.

Applications

RNNs are used in natural language processing, speech recognition, and time series prediction tasks.



Deep Learning Architectures

Long Short-Term Memory (LSTM) Networks

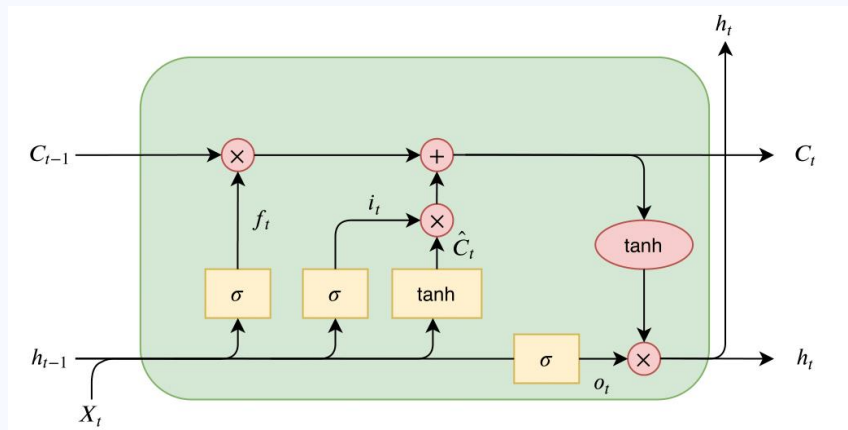
Long Short-Term Memory (LSTM) networks are a type of RNN architecture designed to overcome the vanishing gradient problem and capture long-term dependencies in sequential data.

Structure

LSTMs have a more complex architecture than traditional RNNs, including gates that control the flow of information.

Applications

LSTMs are commonly used in tasks such as speech recognition, language translation, and sentiment analysis.



Deep Learning Architectures

Comparison of Architectures

CNNs

Suited for processing grid-like data, such as images.

RNNs

Ideal for handling sequential data with temporal dependencies.

LSTMs

Effective for capturing long-term dependencies in sequential data while avoiding the vanishing gradient problem.

Deep Learning Architectures

Applications of Deep Learning Architectures

Image Recognition

CNNs are used for tasks like object detection, image classification, and facial recognition.

Natural Language Processing

RNNs and LSTMs are applied in tasks such as language translation, sentiment analysis, and text generation.

Time Series Analysis

RNNs and LSTMs are employed for tasks like stock price prediction, weather forecasting, and anomaly detection.

Deep Learning Architectures

SUMMARY

- ✓ Deep learning architectures such as CNNs, RNNs, and LSTMs are designed to handle different types of data and problem domains.
- ✓ Each architecture has specific structures and functionalities tailored to its intended applications.
- ✓ Understanding these architectures provides insights into their roles and suitability for solving various real-world problems.

5.4

Practical Deep Learning

Lesson Learning Outcomes

- ✓ Gain familiarity with popular deep learning frameworks such as TensorFlow, PyTorch, and Keras
- ✓ Understand the steps involved in building and training a deep learning model
- ✓ Learn how to implement a basic deep learning model for a specific task, such as image classification or text generation

LESSON OVERVIEW

Module 5 Practical Deep Learning

Lesson 5.4 Practical Deep Learning

This lesson focuses on the practical aspects of deep learning, including an overview of popular **Deep Learning Frameworks**, **building a deep learning model step-by-step**, and **implementing a basic deep learning model for a specific task**.

We will explore the key features of deep learning frameworks such as **TensorFlow**, **PyTorch**, and **Keras** and learn how to construct and train a neural network for a given problem.

- Deep learning frameworks
- Steps of building a deep learning model
- Implementation examples

Practical Deep Learning

Deep Learning Frameworks

Deep learning frameworks are software libraries that provide **high-level APIs for building, training, and deploying deep learning models**.

Overview of Popular Frameworks:

TensorFlow, **PyTorch**, and **Keras** are among the most widely used deep learning frameworks, offering a range of features and capabilities for developing deep learning models.

Practical Deep Learning

TensorFlow

History

TensorFlow was developed by **Google Brain** and released as an open-source library in **2015**.

It has become one of the **most popular deep learning frameworks** due to its **flexibility, scalability, and extensive community support**.

Features

TensorFlow provides a comprehensive ecosystem for building and deploying machine learning and deep learning models, including tools for data preprocessing, model construction, training, and deployment.



TensorFlow

Practical Deep Learning

PyTorch

History

PyTorch is an **open-source deep learning framework** developed by **Facebook's AI Research lab (FAIR)** and released in **2016**.

It is known for its **dynamic computational graph** and ease of use.

Features

PyTorch offers a flexible and intuitive interface for building and training deep learning models, with support for dynamic computation, automatic differentiation, and GPU acceleration.



Practical Deep Learning

Keras

History

Keras is an **open-source deep learning library** written in Python and developed by **François Chollet**.

It was designed to be user-friendly, modular, and extensible, allowing developers to **build complex neural networks with minimal code**.

Features

Keras provides a high-level API for building and training deep learning models, with support for various backends, including TensorFlow and Theano. It is widely used for rapid prototyping and experimentation.



Practical Deep Learning

Building a Deep Learning Model

- Define the model architecture
- Compile the model with an appropriate optimizer and loss function
- Train the model on training data
- Evaluate the model on validation data
- Fine-tune hyperparameters for better performance

Practical Deep Learning

Implementation Example - Load and Preprocess the Dataset

```
# Example code for loading and preprocessing the dataset using TensorFlow
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# Load the MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Preprocess the dataset
X_train = X_train.reshape((X_train.shape[0], 28, 28, 1))
X_test = X_test.reshape((X_test.shape[0], 28, 28, 1))
X_train = X_train.astype('float32') / 255
X_test = X_test.astype('float32') / 255
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
```

Practical Deep Learning

Implementation Example - Define the Neural Network Architecture

```
# Example code for defining the neural network architecture using TensorFlow
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

# Define the model architecture
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(100, activation='relu'),
    Dense(10, activation='softmax')
])
```

Practical Deep Learning

Implementation Example - Compiling, Training and Evaluating the Model

```
# Example code for compiling the model using TensorFlow
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Example code for training the model using TensorFlow
model.fit(X_train, y_train, epochs=10, batch_size=32, validation_split=0.2)

# Example code for evaluating the model using TensorFlow
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print(f'Test Loss: {test_loss}, Test Accuracy: {test_accuracy}')
```

Practical Deep Learning

SUMMARY

- ✓ Practical aspects of deep learning.
- ✓ Overview of deep learning frameworks.
- ✓ Step-by-step guidance on building a deep learning model.

Key Takeaways

SUMMARY

- ✓ Understanding Deep Learning Fundamentals
- ✓ Importance of Deep Learning
- ✓ Understanding of Deep Learning architecture
- ✓ Exploration of different Deep Learning frameworks
- ✓ Understanding the steps for building a Deep Learning model